Database Schema

Communication and Notifications Module

National Freelance & Skill Verification Platform

Team: **ProjectMasters** | Section: **SE-B** | Date: **2026-03-22**

External Data Required from Other Modules

The Communication Module does **not** own user data. The following fields are required from **Module 1 (User Identity, Profile & Portfolio Management)**:

Data Item	Description
`user_id`	Unique identifier of the user (PK in User module)
`name`	Display name in chat UI
`profile_image`	Avatar shown in chat list and message headers
`role`	`freelancer`, `client`, or `admin` — needed for role-based permissions (e.g., only employers create meeting links)
`is_active`	Whether the account is active (to allow/block messaging, for real-time presence)
`email`	For fallback notification delivery

> Note: These fields will referenced via `user_id` foreign key in all tables below but are not stored in the Communication Module's own database.

Tables

1. chat_rooms — Stores both one-to-one and group chat contexts

Column	Type	Constraints	Description
`room_id`	UUID	PK, NOT NULL	Unique chat room identifier
`room_type`	ENUM('direct','group')	NOT NULL	Type of chat room
`room_name`	VARCHAR(100)	NULLABLE	Only for group chats
`created_by`	UUID	FK → user_id (external)	User who created the room
`created_at`	TIMESTAMP	NOT NULL, DEFAULT NOW()	Room creation time
`is_active`	BOOLEAN	DEFAULT TRUE	Soft delete flag

Indexes: `room_type`, `created_by`

2. chat_room_members — Junction table — maps users to chat rooms

Column	Type	Constraints	Description
membership_id	UUID	PK, NOT NULL	Unique membership record
room_id	UUID	FK → chat_rooms(room_id), NOT NULL	Associated chat room
user_id	UUID	FK → user_id (external), NOT NULL	Member's user ID
member_role	ENUM('admin','member')	DEFAULT 'member'	Role within the group (CM-FR-16/18)
joined_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	When user joined
is_active	BOOLEAN	DEFAULT TRUE	Allows removing members without hard delete

Unique: (`room_id`*,* `user_id`*)* | *Indexes:* `room_id`*,* `user_id`

3. messages — Core table storing all messages across all chat rooms

Column	Type	Constraints	Description
msg_id	UUID	PK, NOT NULL	Unique message identifier
room_id	UUID	FK → chat_rooms(room_id), NOT NULL	Which chat room this belongs to
sender_id	UUID	FK → user_id (external), NOT NULL	Who sent the message
message_type	ENUM('text','media','meeting_link','system')	NOT NULL, DEFAULT 'text'	Type of message content
content	TEXT	NULLABLE	Text body (null if pure media)
media_id	UUID	FK → media_files(media_id), NULLABLE	Attached media, if any
meeting_link_id	UUID	FK → meeting_links(link_id), NULLABLE	Attached meeting link, if any
status	ENUM('sent','delivered','read','failed')	DEFAULT 'sent'	Delivery status (CM-FR-4)
sent_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Message timestamp (CM-FR-3)
is_deleted	BOOLEAN	DEFAULT FALSE	Soft delete for message history
retry_count	SMALLINT	DEFAULT 0	Tracks retry attempts (CM-FR-5)

Indexes: `room_id`*,* `sender_id`*,* `sent_at` *(for sorted chat history fetch)*

4. message_receipts — Per-user read/delivery status in group chats

Column	Type	Constraints	Description
receipt_id	UUID	PK, NOT NULL	Unique receipt record
msg_id	UUID	FK → messages(msg_id), NOT NULL	The message
recipient_id	UUID	FK → user_id (external), NOT NULL	The recipient
delivered_at	TIMESTAMP	NULLABLE	When delivered to recipient's device
read_at	TIMESTAMP	NULLABLE	When recipient opened/read message

Unique: (msg_id, recipient_id) | *Indexes:* msg_id, recipient_id

5. media_files — Metadata for all uploaded files — actual binary is in cloud storage

Column	Type	Constraints	Description
media_id	UUID	PK, NOT NULL	Unique media record
uploader_id	UUID	FK → user_id (external), NOT NULL	Who uploaded the file
room_id	UUID	FK → chat_rooms(room_id), NOT NULL	Associated chat context
file_name	VARCHAR(255)	NOT NULL	Original file name
file_type	ENUM('image','video','pdf','doc','other')	NOT NULL	Validated type (CM-FR-8)
mime_type	VARCHAR(100)	NOT NULL	e.g., image/jpeg, application/pdf
file_size_bytes	BIGINT	NOT NULL	For size limit enforcement (CM-FR-9)
storage_url	TEXT	NOT NULL	Cloud storage URL
uploaded_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Upload timestamp
is_deleted	BOOLEAN	DEFAULT FALSE	Soft delete

Indexes: uploader_id, room_id, file_type

6. meeting_links — Generated meeting links shared via chat (CM-FR-10, 11, 12)

Column	Type	Constraints	Description
link_id	UUID	PK, NOT NULL	Unique meeting link record
created_by	UUID	FK → user_id (external), NOT NULL	Only employers/admins per SRS
room_id	UUID	FK → chat_rooms(room_id), NOT NULL	Chat room where link was shared
platform	ENUM('zoom','google_meet')	NOT NULL	Third-party platform (CM-FR-11)

Column	Type	Constraints	Description
`meeting_url`	TEXT	NOT NULL	The generated meeting URL
`scheduled_at`	TIMESTAMP	NULLABLE	Optional scheduled time
`created_at`	TIMESTAMP	NOT NULL, DEFAULT NOW()	When link was generated
`is_expired`	BOOLEAN	DEFAULT FALSE	For link lifecycle management

Indexes: `room_id`, `created_by`

7. notifications — All in-app notification records per user (CM-FR-13, 14)

Column	Type	Constraints	Description
`notif_id`	UUID	PK, NOT NULL	Unique notification ID
`recipient_id`	UUID	FK → user_id (external), NOT NULL	Target user
`notif_type`	ENUM('new_message','file_received','meeting_link','group_added','group_removed')	NOT NULL	Event type
`related_room_id`	UUID	FK → chat_rooms(room_id), NULLABLE	Associated chat room
`related_msg_id`	UUID	FK → messages(msg_id), NULLABLE	Associated message
`content`	VARCHAR(255)	NOT NULL	Short notification text
`is_read`	BOOLEAN	DEFAULT FALSE	Read/unread status
`created_at`	TIMESTAMP	NOT NULL, DEFAULT NOW()	When notification was generated

Indexes: `recipient_id`, `is_read`, `created_at`

Key Design Decisions

Decision	Rationale
UUID primary keys throughout	Avoids sequential ID guessing; safer for distributed/microservice setup
Soft deletes (`is_deleted`) on messages and media	Preserves chat history integrity; supports dispute resolution (per SRS §5.2)
`message_receipts` as separate table	Necessary for group read receipts — a single `status` column on `messages` only works for 1:1
`message_type` enum on `messages`	Allows a single unified messages table rather than separate tables per content type
`media_files` separate from `messages`	Media metadata is heavy; keeping it separate improves message query performance
No user data stored locally	Full compliance with the microservices architecture described in SRS §2.1
`meeting_links` stored separately	Enables history retrieval (CM-FR-12) and expiry management independently of messages
MoSCoW alignment	Schema fully covers all Must Have features; voice_message (Could Have, per RMP §4.3) is omitted — out of current scope

SQL DDL — CREATE TABLE Statements

The following DDL is written for **PostgreSQL**. ENUM types are expressed as **VARCHAR + CHECK constraints** for maximum portability. UUID generation uses `gen_random_uuid()` from the `pgcrypto` extension.

```
-- Enable UUID extension (PostgreSQL)

-- ─── 1. chat_rooms ──────────────────────────────────────────
CREATE TABLE chat_rooms (
    room_id     UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    room_type   VARCHAR(10) NOT NULL CHECK (room_type IN ('direct','group')),
    room_name   VARCHAR(100),
    created_by  UUID        NOT NULL,   -- FK → user_id (external)
    created_at  TIMESTAMP   NOT NULL DEFAULT NOW(),
    is_active   BOOLEAN     NOT NULL DEFAULT TRUE
);
CREATE INDEX idx_chatrooms_type      ON chat_rooms (room_type);
CREATE INDEX idx_chatrooms_createdby ON chat_rooms (created_by);

-- ─── 2. chat_room_members ───────────────────────────────────
CREATE TABLE chat_room_members (
    membership_id UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    room_id       UUID        NOT NULL REFERENCES chat_rooms(room_id) ON DELETE CASCADE,
    user_id       UUID        NOT NULL,   -- FK → user_id (external)
    member_role   VARCHAR(10) NOT NULL DEFAULT 'member'
                              CHECK (member_role IN ('admin','member')),
    joined_at     TIMESTAMP   NOT NULL DEFAULT NOW(),
    is_active     BOOLEAN     NOT NULL DEFAULT TRUE,
    UNIQUE (room_id, user_id)
);
CREATE INDEX idx_members_room ON chat_room_members (room_id);
CREATE INDEX idx_members_user ON chat_room_members (user_id);
```

```
-- ─── 3. media_files ──────────────────────────────────────────
CREATE TABLE media_files (
    media_id        UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
    uploader_id     UUID         NOT NULL,   -- FK → user_id (external)
    room_id         UUID         NOT NULL REFERENCES chat_rooms(room_id),
    file_name       VARCHAR(255) NOT NULL,
    file_type       VARCHAR(10)  NOT NULL
                    CHECK (file_type IN ('image','video','pdf','doc','other')),
    mime_type       VARCHAR(100) NOT NULL,
    file_size_bytes BIGINT       NOT NULL,
    storage_url     TEXT         NOT NULL,
    thumbnail_url   TEXT,
    uploaded_at     TIMESTAMP    NOT NULL DEFAULT NOW(),
    is_deleted      BOOLEAN      NOT NULL DEFAULT FALSE
);
CREATE INDEX idx_media_uploader ON media_files (uploader_id);
CREATE INDEX idx_media_room     ON media_files (room_id);
CREATE INDEX idx_media_type     ON media_files (file_type);

-- ─── 4. meeting_links ────────────────────────────────────────
CREATE TABLE meeting_links (
    link_id      UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    created_by   UUID        NOT NULL,   -- FK → user_id (external)
    room_id      UUID        NOT NULL REFERENCES chat_rooms(room_id),
    platform     VARCHAR(20) NOT NULL
                 CHECK (platform IN ('zoom','google_meet','other')),
    meeting_url  TEXT        NOT NULL,
    scheduled_at TIMESTAMP,
    created_at   TIMESTAMP   NOT NULL DEFAULT NOW(),
    is_expired   BOOLEAN     NOT NULL DEFAULT FALSE
);
CREATE INDEX idx_mlinks_room      ON meeting_links (room_id);
CREATE INDEX idx_mlinks_createdby ON meeting_links (created_by);

-- ─── 5. messages ─────────────────────────────────────────────
CREATE TABLE messages (
    msg_id          UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    room_id         UUID        NOT NULL REFERENCES chat_rooms(room_id),
    sender_id       UUID        NOT NULL,   -- FK → user_id (external)
    message_type    VARCHAR(15) NOT NULL DEFAULT 'text'
                    CHECK (message_type IN ('text','media','meeting_link','system')),
    content         TEXT,
    media_id        UUID        REFERENCES media_files(media_id),
    meeting_link_id UUID        REFERENCES meeting_links(link_id),
    status          VARCHAR(10) NOT NULL DEFAULT 'sent'
                    CHECK (status IN ('sent','delivered','read','failed')),
    sent_at         TIMESTAMP   NOT NULL DEFAULT NOW(),
    is_deleted      BOOLEAN     NOT NULL DEFAULT FALSE,
    retry_count     SMALLINT    NOT NULL DEFAULT 0
);
CREATE INDEX idx_messages_room     ON messages (room_id);
CREATE INDEX idx_messages_sender   ON messages (sender_id);
CREATE INDEX idx_messages_sent_at  ON messages (sent_at);

-- ─── 6. message_receipts ─────────────────────────────────────
CREATE TABLE message_receipts (
```

```
    receipt_id   UUID      PRIMARY KEY DEFAULT gen_random_uuid(),
    msg_id       UUID      NOT NULL REFERENCES messages(msg_id) ON DELETE CASCADE,
    recipient_id UUID      NOT NULL,   -- FK → user_id (external)
    delivered_at TIMESTAMP,
    read_at      TIMESTAMP,
    UNIQUE (msg_id, recipient_id)
);
CREATE INDEX idx_receipts_msg       ON message_receipts (msg_id);
CREATE INDEX idx_receipts_recipient ON message_receipts (recipient_id);

-- ─── 7. notifications ──────────────────────────────────────
CREATE TABLE notifications (
    notif_id        UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
    recipient_id    UUID         NOT NULL,   -- FK → user_id (external)
    notif_type      VARCHAR(20)  NOT NULL
                    CHECK (notif_type IN
                      ('new_message','file_received','meeting_link',
                       'group_added','group_removed')),
    related_room_id UUID         REFERENCES chat_rooms(room_id),
    related_msg_id  UUID         REFERENCES messages(msg_id),
    content         VARCHAR(255) NOT NULL,
    is_read         BOOLEAN      NOT NULL DEFAULT FALSE,
    created_at      TIMESTAMP    NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_notifs_recipient ON notifications (recipient_id);
CREATE INDEX idx_notifs_is_read   ON notifications (is_read);
CREATE INDEX idx_notifs_created   ON notifications (created_at);
```